

Machine Learning Algorithms

Every Data Scientist Must Know

A Visual, Intuitive Guide with Math Foundations

Zamir Hussain
Department of Mathematics
University of Wah

February 9, 2026

Abstract

These notes bridge core linear algebra concepts with practical machine learning algorithms. Designed for mathematics students new to ML, each section provides intuitive explanations, concrete numerical examples, and clear connections to underlying mathematical principles. Emphasis is placed on "why this works" rather than formal proofs.

Contents

1	Introduction: The Big Picture [Easy]	3
1.1	The Core Recipe	3
1.2	Types of Machine Learning	3
2	Classification: Predicting Categories [Medium]	3
2.1	Logistic Regression: Probability Estimator [Easy]	4
2.2	Naive Bayes: Probabilistic Detective [Easy]	5
2.3	K-Nearest Neighbors (KNN): Lazy Learner [Easy]	8
2.4	Support Vector Machine (SVM): Maximum Margin [Advanced]	9
2.5	Decision Tree: If-Else Rules [Easy]	11
2.6	Random Forest: Ensemble Wisdom [Medium]	13
3	Regression: Predicting Numbers [Medium]	14
3.1	Linear Regression: Best Fit Line [Easy]	14
3.2	Ridge Regression: L2 Regularization [Medium]	16
3.3	Lasso Regression: L1 Regularization [Medium]	17
4	Dimensionality Reduction: Compress Data [Advanced]	18
4.1	PCA: Find Important Directions [Medium]	18
4.2	ICA: Separate Mixed Signals [Advanced]	20

5	Association Rule Mining: Find Patterns [Medium]	21
5.1	Apriori: Frequent Itemsets [Medium]	21
6	Anomaly Detection: Find Outliers [Medium]	23
6.1	Z-Score: Statistical Method [Easy]	23
6.2	Isolation Forest: Isolate Anomalies [Medium]	23
7	Semi-Supervised Learning: Use Unlabeled Data [Advanced]	24
7.1	Self-Training: Bootstrap [Medium]	24
7.2	Co-Training: Two Views [Advanced]	24
8	Reinforcement Learning: Learn by Doing [Advanced]	25
8.1	Q-Learning: Value Iteration [Advanced]	25
8.2	Policy Gradient: Direct Optimization [Advanced]	26

1 Introduction: The Big Picture [Easy]

💡 Intuition

Machine Learning is teaching computers to recognize patterns using math. Think of it as finding the "rule" behind data without explicitly programming it.

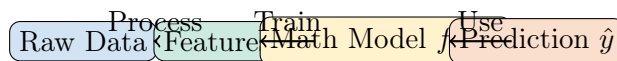
☕ Everyday Analogy

Learning to Ride a Bike:

- **Traditional Programming:** Write 10,000 rules (if leaning left, push right pedal...)
- **Machine Learning:** Show 1000 examples of riding/falling, let computer figure out the balance rules

1.1 The Core Recipe

Every ML algorithm follows this pattern:



📊 The Math (Step-by-Step)

The Universal Equation Behind Most ML:

$$\text{Prediction} = \text{Activation}(\text{Weights} \times \text{Data} + \text{Bias})$$

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x} + b)$$

What Each Symbol Means:

- **x:** Your data as a vector (e.g., [height, weight, age])
- **w:** Weights the model learns (importance of each feature)
- **b:** Bias (starting point, like y-intercept)
- **f(·):** Activation function (decision maker)

1.2 Types of Machine Learning

Type	Data	Goal	Example
Supervised	$(\mathbf{x}, y)$ pairs	Learn $f : \mathbf{x} \rightarrow y$	Predict house prices
Unsupervised	Only $\mathbf{x}$	Find hidden structure	Group customers
Semi-Supervised	Few $(\mathbf{x}, y)$, many $\mathbf{x}$	Use unlabeled to help	Medical images
Reinforcement	States, Actions, Rewards	Maximize reward	Game playing

2 Classification: Predicting Categories [Medium]

2.1 Logistic Regression: Probability Estimator [Easy]

💡 Intuition

Instead of predicting "Yes/No" directly, predict the **probability** of Yes. Use a special "S-curve" (sigmoid) to squeeze any number into 0-1 range.

☕ Everyday Analogy

Thermostat Analogy:

- Temperature reading (any number) → Sigmoid → "Heat On" probability
- Very cold → 99% chance heat is on
- Very hot → 1% chance heat is on
- Comfortable → 50% (uncertain)

📊 The Math (Step-by-Step)

Step 1: Linear Combination (like Linear Regression)

$$z = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = \mathbf{w}^\top \mathbf{x} + b$$

Step 2: Sigmoid Function (The "Squeeze")

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- $\sigma(0) = 0.5$ (uncertain)
- $\sigma(+\infty) = 1$ (definitely class 1)
- $\sigma(-\infty) = 0$ (definitely class 0)

Step 3: Decision Rule

$$\text{Predict } \begin{cases} \text{Class 1} & \text{if } \sigma(z) \geq 0.5 \\ \text{Class 0} & \text{if } \sigma(z) < 0.5 \end{cases}$$

🔗 Worked Example

Email Spam Detection:

Features: [contains "free", contains "money", length > 100 chars, has attachment]

Email: ["free"]=1, "money"]=0, length=1, attachment=0]

Learned weights: $w_1 = 2.5, w_2 = 1.8, w_3 = -0.5, w_4 = -1.2$, bias $b = -1.0$

Calculation:

$$z = 2.5(1) + 1.8(0) + (-0.5)(1) + (-1.2)(0) + (-1.0) = 2.5 - 0.5 - 1.0 = 1.0$$

$$\text{Probability} = \sigma(1.0) = \frac{1}{1 + e^{-1}} = \frac{1}{1 + 0.368} = 0.73$$

Result: 73% spam probability → Classify as Spam

Python Implementation

```
1 import numpy as np
2
3 def sigmoid(z):
4     return 1 / (1 + np.exp(-z))
5
6 # Data: [free, money, long, attachment]
7 x = np.array([1, 0, 1, 0])
8 w = np.array([2.5, 1.8, -0.5, -1.2])
9 b = -1.0
10
11 z = np.dot(w, x) + b
12 prob = sigmoid(z)
13 prediction = 1 if prob >= 0.5 else 0
```

Common Pitfalls

- **Don't use for non-binary problems directly** (use Softmax for 3+ classes)
- **Assumes linear decision boundary** (use kernels or neural nets for complex shapes)
- **Sensitive to outliers** (extreme values can skew the sigmoid)

2.2 Naive Bayes: Probabilistic Detective [Easy]

Intuition

Use Bayes' Theorem "backwards": Update your belief about a class as you see evidence. "Naive" = assume all features are independent (simplifies math massively).

Everyday Analogy

Medical Diagnosis:

- Prior: 1% of population has disease
- Evidence: Positive test result
- Naive Bayes: Update to 15% probability (Bayes' Theorem)
- Add symptom: Fever → Update to 60%
- Add symptom: Cough → Update to 90%

The Math (Step-by-Step)

Bayes' Theorem (The Core):

$$P(\text{Disease}|\text{Symptoms}) = \frac{P(\text{Disease}) \times P(\text{Symptoms}|\text{Disease})}{P(\text{Symptoms})}$$

Naive Assumption (Independence):

$$P(\text{Symptoms}|\text{Disease}) = P(\text{Fever}|\text{Disease}) \times P(\text{Cough}|\text{Disease}) \times \dots$$

Decision Rule (Pick the class with highest score):

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^d P(x_i|y)$$

Types:

- **Gaussian:** Features are continuous numbers (heights, weights)
- **Multinomial:** Features are counts (word frequencies)
- **Bernoulli:** Features are binary (word present/absent)

Worked Example

Play Tennis? (Weather Dataset)

Training Data:

Outlook	Temp	Humidity	Wind	Play?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

New Day: [Sunny, Cool, High, Strong] - Will we play?

Step 1: Calculate Priors

$$P(\text{Yes}) = \frac{9}{14} = 0.64, \quad P(\text{No}) = \frac{5}{14} = 0.36$$

Step 2: Calculate Likelihoods

$$\begin{aligned} P(\text{Sunny}|\text{Yes}) &= \frac{2}{9} = 0.22 & P(\text{Sunny}|\text{No}) &= \frac{3}{5} = 0.60 \\ P(\text{Cool}|\text{Yes}) &= \frac{3}{9} = 0.33 & P(\text{Cool}|\text{No}) &= \frac{1}{5} = 0.20 \\ P(\text{High}|\text{Yes}) &= \frac{3}{9} = 0.33 & P(\text{High}|\text{No}) &= \frac{4}{5} = 0.80 \\ P(\text{Strong}|\text{Yes}) &= \frac{3}{9} = 0.33 & P(\text{Strong}|\text{No}) &= \frac{3}{5} = 0.60 \end{aligned}$$

Step 3: Multiply

$$P(\text{Yes}|\mathbf{x}) \propto 0.64 \times 0.22 \times 0.33 \times 0.33 \times 0.33 = 0.005$$

$$P(\text{No}|\mathbf{x}) \propto 0.36 \times 0.60 \times 0.20 \times 0.80 \times 0.60 = 0.021$$

Result: $0.021 > 0.005 \rightarrow$ Predict **No** (Don't play tennis)

🔍 Why This Matters

Why Use Naive Bayes?

- **Extremely fast** (just counting and multiplying)
- **Works well with high dimensions** (many features)
- **Needs little training data**
- **Handles missing values naturally**
- **Great baseline** before trying complex models

2.3 K-Nearest Neighbors (KNN): Lazy Learner [Easy]

💡 Intuition

No training phase! Just store all data. When predicting, find the k most similar past examples and vote.

☕ Everyday Analogy

Restaurant Recommendation:

- You: "I want Italian food, \$20 budget, romantic atmosphere"
- KNN: Finds 5 most similar restaurants you've liked before
- If 4 were Italian and 1 was French → Recommend Italian

📊 The Math (Step-by-Step)

Distance = Similarity

Euclidean Distance (Straight line):

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_d - b_d)^2} = \|\mathbf{a} - \mathbf{b}\|_2$$

Manhattan Distance (City blocks):

$$d(\mathbf{a}, \mathbf{b}) = |a_1 - b_1| + |a_2 - b_2| + \dots + |a_d - b_d| = \|\mathbf{a} - \mathbf{b}\|_1$$

Algorithm:

1. Calculate distance from new point to ALL training points
2. Sort by distance (ascending)
3. Pick top k nearest neighbors
4. Majority vote (classification) or average (regression)

Worked Example

Classify Fruit: [Weight=150g, Color=Red]

Training Data:

Fruit	Weight	Color Score	Class
A	140g	0.8 (Red)	Apple
B	130g	0.9 (Red)	Apple
C	160g	0.7 (Red)	Apple
D	300g	0.3 (Yellow)	Banana
E	320g	0.4 (Yellow)	Banana

Distances from New Point (150, 0.75):

$$d(A) = \sqrt{(150 - 140)^2 + (0.75 - 0.8)^2} = \sqrt{100 + 0.0025} \approx 10.0$$

$$d(B) = \sqrt{(150 - 130)^2 + (0.75 - 0.9)^2} = \sqrt{400 + 0.0225} \approx 20.0$$

$$d(C) = \sqrt{(150 - 160)^2 + (0.75 - 0.7)^2} = \sqrt{100 + 0.0025} \approx 10.0$$

$$d(D) = \sqrt{(150 - 300)^2 + (0.75 - 0.3)^2} = \sqrt{22500 + 0.2025} \approx 150.0$$

$$d(E) = \sqrt{(150 - 320)^2 + (0.75 - 0.4)^2} = \sqrt{28900 + 0.1225} \approx 170.0$$

K=3: Nearest are A (Apple), C (Apple), B (Apple)

Vote: 3 Apples, 0 Bananas → Predict **Apple**



Common Pitfalls

- **Curse of Dimensionality:** Distance becomes meaningless in high dimensions (100+ features)
- **Sensitive to scale:** Weight (kg) vs Height (cm) - normalize features!
- **Slow prediction:** Must compare to all training points (use KD-trees or Ball trees for speedup)
- **Choosing k:** Too small = noise sensitive, Too large = over-smoothing

2.4 Support Vector Machine (SVM): Maximum Margin [Advanced]

Intuition

Find the "widest street" (margin) that separates classes. The boundary is determined only by the closest points (support vectors).

Everyday Analogy

Separating Two Groups:

- **Bad:** Line close to one group (risk of misclassification)
- **Good:** Line in the middle with maximum breathing room
- **SVM:** Find the line with maximum margin (distance to nearest points)

The Math (Step-by-Step)

The Margin Concept:

For a line $\mathbf{w}^\top \mathbf{x} + b = 0$:

- **Margin width** = $\frac{2}{\|\mathbf{w}\|}$ (distance between parallel boundary lines)
- **Goal:** Maximize margin $\Leftrightarrow$ Minimize $\|\mathbf{w}\|$

Optimization Problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$$

Kernel Trick (for non-linear boundaries): Instead of $\mathbf{x}$, use $\phi(\mathbf{x})$ (higher dimensional mapping)

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

Common Kernels:

- Linear: $K(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v}$
- Polynomial: $K(\mathbf{u}, \mathbf{v}) = (\mathbf{u}^\top \mathbf{v} + c)^d$
- RBF (Gaussian): $K(\mathbf{u}, \mathbf{v}) = \exp(-\gamma \|\mathbf{u} - \mathbf{v}\|^2)$

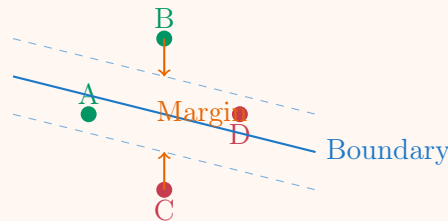
Worked Example

2D Linear SVM:

Data:

Point	Coordinates	Class
A	(1, 2)	+1
B	(2, 3)	+1
C	(2, 1)	-1
D	(3, 2)	-1

Visual:



Optimal boundary: $x_1 - x_2 = 0$ (45° line)

Support vectors: B and C (closest to boundary)

Margin width: $\sqrt{2}$

Why This Matters

Why SVM?

- **Effective in high dimensions** (especially with kernels)
- **Memory efficient:** Only uses support vectors
- **Versatile:** Different kernels for different data types
- **Robust:** Maximizing margin reduces overfitting

2.5 Decision Tree: If-Else Rules [Easy]

Intuition

Ask a series of yes/no questions to narrow down the answer. Each question splits the data into purer groups.

Everyday Analogy

20 Questions Game:

- Q1: Is it an animal? → Yes
- Q2: Does it fly? → No
- Q3: Is it a mammal? → Yes
- Q4: Does it live in water? → Yes → Whale!

The Math (Step-by-Step)

Measuring Purity:

Gini Impurity (probability of wrong classification):

$$G = 1 - \sum_{i=1}^C (p_i)^2$$

where p_i = proportion of class i in the node.

Entropy (measure of disorder):

$$H = - \sum_{i=1}^C p_i \log_2(p_i)$$

Information Gain (how much a split helps):

$$IG = H_{\text{parent}} - \sum_j \frac{|D_j|}{|D|} H(D_j)$$

Algorithm:

1. Calculate IG for every possible split
2. Choose split with highest IG
3. Repeat recursively until stopping condition

Worked Example

Play Tennis Decision:

Dataset:

Outlook	Temp	Humidity	Wind	Play?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes

Step 1: Calculate parent entropy

$$H_{\text{parent}} = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

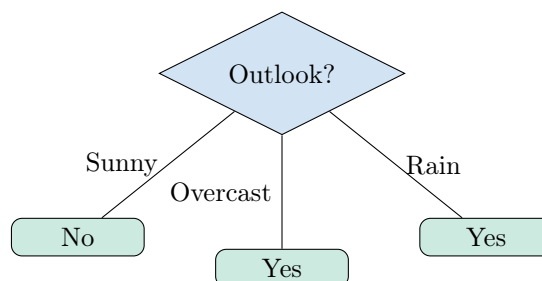
Step 2: Try splitting on Outlook

- Sunny: 2 No, 0 Yes $\rightarrow H = 0$
- Overcast: 0 No, 1 Yes $\rightarrow H = 0$
- Rain: 0 No, 2 Yes $\rightarrow H = 0$

$$H_{\text{children}} = \frac{2}{5}(0) + \frac{1}{5}(0) + \frac{2}{5}(0) = 0$$

$$IG = 0.971 - 0 = 0.971 \text{ (Perfect split!)}$$

Result: Split on Outlook first.



Common Pitfalls

- **Overfitting:** Trees grow until perfect fit $\rightarrow$ Use max_depth, min_samples_split
- **Unstable:** Small data changes $\rightarrow$ Completely different tree
- **Bias:** Favors features with more levels
- **Solution:** Use Random Forest (many trees voting)

2.6 Random Forest: Ensemble Wisdom [Medium]

Intuition

Instead of one decision tree, grow 100+ trees on random subsets of data and features. Let them vote.

☕ Everyday Analogy

Jury Trial:

- One juror = One tree (might be biased)
- Full jury = Random Forest (diverse perspectives)
- Verdict = Majority vote (more reliable)

📊 The Math (Step-by-Step)

Bagging (Bootstrap Aggregating):

Step 1: Bootstrap Sampling

- Original data: N samples
- New dataset: Randomly pick N samples with replacement
- Some samples appear multiple times, some never (out-of-bag)

Step 2: Random Feature Selection

- At each split, consider only $\sqrt{d}$ random features
- Forces diversity among trees

Step 3: Aggregation

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), T_2(\mathbf{x}), \dots, T_B(\mathbf{x})\}$$

🔗 Worked Example

100-Tree Forest Prediction:

For input $\mathbf{x}$, individual tree predictions:

T1	T2	T3	T4	T5	...	T98	T99	T100	Final
A	B	A	A	B	...	A	B	A	

Count: A=65, B=35 → Predict **A**

Out-of-Bag Error:

- Each tree sees ~63% of data (bootstrap)
- Remaining 37% = OOB samples (natural validation set)
- Average OOB error = estimate of test error

3 Regression: Predicting Numbers [Medium]

3.1 Linear Regression: Best Fit Line [Easy]

💡 Intuition

Find the line that minimizes the total squared distance to all points. Like finding the "center of gravity" for the data cloud.

Everyday Analogy

Archery:

- Each arrow = data point
- Target line = your prediction
- Linear regression = move target to minimize average squared distance from arrows

The Math (Step-by-Step)

The Loss Function (Mean Squared Error):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Matrix Form:

$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$$

Closed-Form Solution (Normal Equations):

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Geometric Interpretation: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$ is the projection of $\mathbf{y}$ onto the column space of $\mathbf{X}$.
Gradient Descent (when n is large):

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{2}{n} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Worked Example

House Price Prediction:

Data:

Size (sqft)	Price (\$1000s)
1000	200
1500	300
2000	400
2500	500

Design Matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1000 \\ 1 & 1500 \\ 1 & 2000 \\ 1 & 2500 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 200 \\ 300 \\ 400 \\ 500 \end{bmatrix}$$

Calculate:

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 4 & 7000 \\ 7000 & 13,500,000 \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \begin{bmatrix} 1.35 & -0.0007 \\ -0.0007 & 0.0000004 \end{bmatrix}$$

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 1400 \\ 2,750,000 \end{bmatrix}$$

Solution:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \begin{bmatrix} 0 \\ 0.2 \end{bmatrix}$$

Model: Price = 0 + 0.2 × Size = \$200 per sqft



3.2 Ridge Regression: L2 Regularization [Medium]

Intuition

Add a penalty for large weights. Like adding "springs" to weights to keep them small and prevent overfitting.

The Math (Step-by-Step)

Modified Loss Function:

$$J(\mathbf{w}) = \underbrace{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2}_{\text{Fit}} + \underbrace{\lambda \|\mathbf{w}\|^2}_{\text{Penalty}}$$

Solution:

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Effect:

- $\lambda = 0$: Regular linear regression
- $\lambda \rightarrow \infty$: All weights $\rightarrow 0$
- λ in between: Shrinks weights proportionally

SVD Interpretation: If $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, then ridge shrinks singular values:

$$\sigma_i \rightarrow \frac{\sigma_i}{\sigma_i^2 + \lambda}$$

Worked Example

Multicollinearity Example:

Predicting house price using:

- x_1 = Size in sqft
- x_2 = Size in sqm (perfectly correlated with x_1)

Without Ridge: Unstable! Small data changes $\rightarrow$ huge weight changes

$$w_1 = 500, w_2 = -500 \text{ (cancels out)}$$

With Ridge ($\lambda = 1$):

$$w_1 = 0.2, w_2 = 0.2 \text{ (stable, shared importance)}$$

Bias-Variance Tradeoff:

- λ small: Low bias, high variance (overfitting)
- λ large: High bias, low variance (underfitting)

3.3 Lasso Regression: L1 Regularization [Medium]

Intuition

Like Ridge, but can zero out unimportant features completely. Automatic feature selection.

The Math (Step-by-Step)

Modified Loss Function:

$$J(\mathbf{w}) = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \sum_{j=1}^d |w_j|$$

No Closed Form (absolute value not differentiable at 0)

Coordinate Descent Solution: For each weight w_j :

$$w_j = \begin{cases} z_j - \lambda & \text{if } z_j > \lambda \\ 0 & \text{if } |z_j| \leq \lambda \\ z_j + \lambda & \text{if } z_j < -\lambda \end{cases}$$

where z_j is the partial residual.

Geometric Interpretation: Lasso constraint ($\|\mathbf{w}\|_1 \leq t$) has corners at axes $\rightarrow$ sparse solutions.

Worked Example

Feature Selection:

10 features, only 3 are relevant:

Ridge ($\lambda = 1$):

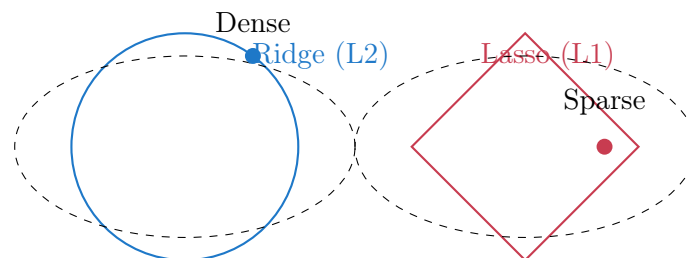
$$\mathbf{w} = [0.8, 0.1, 0.9, 0.05, 0.02, 0.01, 0.03, 0.04, 0.02, 0.01]$$

All non-zero (hard to interpret)

Lasso ($\lambda = 0.5$):

$$\mathbf{w} = [0.75, 0, 0.85, 0, 0, 0, 0, 0, 0, 0]$$

Only 3 features selected automatically!



4 Dimensionality Reduction: Compress Data [Advanced]

4.1 PCA: Find Important Directions [Medium]

Intuition

Rotate your data to find the directions with maximum spread (variance). Keep only the most spread-out directions.

Everyday Analogy

Photography:

- 3D world $\rightarrow$ 2D photo loses depth but keeps main features
- PCA: Find the angle that shows the most "spread" of data
- Keep top 2-3 angles, discard the rest

The Math (Step-by-Step)

Algorithm Steps:

1. **Center data:** $\tilde{\mathbf{X}} = \mathbf{X} - \text{mean}(\mathbf{X})$
2. **Compute covariance:** $\mathbf{C} = \frac{1}{n-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$
3. **Eigen decomposition:** $\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i$
4. **Sort:** $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$
5. **Project:** $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{V}_k$ (first k eigenvectors)

Variance Retained:

$$\text{Ratio} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

SVD Approach (more stable):

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \Rightarrow \mathbf{Z} = \mathbf{U}_k\mathbf{\Sigma}_k$$

Worked Example

2D to 1D:

$$\text{Data: } \mathbf{X} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{bmatrix} \text{ (perfect correlation)}$$

Step 1: Center

$$\tilde{\mathbf{X}} = \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix}$$

Step 2: Covariance

$$\mathbf{C} = \frac{1}{3} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} = \begin{bmatrix} 1.67 & 1.67 \\ 1.67 & 1.67 \end{bmatrix}$$

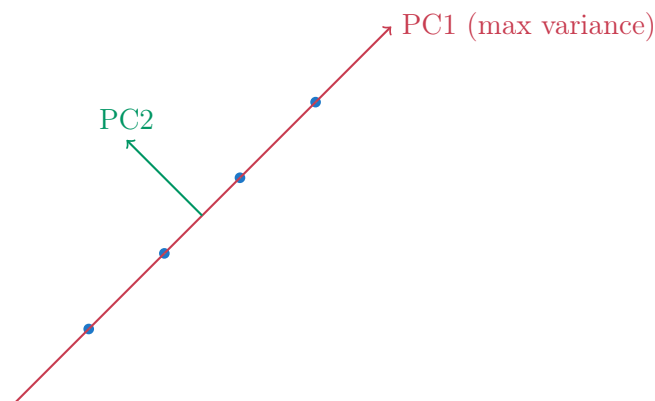
Step 3: Eigenvalues/vectors

$$\lambda_1 = 3.33, \mathbf{v}_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}, \quad \lambda_2 = 0, \mathbf{v}_2 = \begin{bmatrix} -0.707 \\ 0.707 \end{bmatrix}$$

Step 4: Project

$$\mathbf{z} = \tilde{\mathbf{X}} \mathbf{v}_1 = \begin{bmatrix} -2.12 \\ -0.71 \\ 0.71 \\ 2.12 \end{bmatrix}$$

Result: 100% variance retained (data was 1D in 2D space)



4.2 ICA: Separate Mixed Signals [Advanced]

Intuition

"Cocktail party problem": Separate mixed audio sources. Assumes sources are statistically independent and non-Gaussian.

Everyday Analogy

Cocktail Party:

- 3 people talking, 3 microphones recording mixtures
- ICA: Separate the 3 voices from the mixtures
- Key assumption: Voices are independent (one person talking doesn't predict another)

The Math (Step-by-Step)

Linear Mixing Model:

$$\mathbf{x} = \mathbf{A}\mathbf{s}$$

where $\mathbf{x}$ = observed mixtures, $\mathbf{A}$ = mixing matrix, $\mathbf{s}$ = independent sources.

Goal: Find unmixing matrix $\mathbf{W} \approx \mathbf{A}^{-1}$ such that:

$$\mathbf{y} = \mathbf{W}\mathbf{x} \approx \mathbf{s}$$

Key Principle: Maximize non-Gaussianity of $\mathbf{y}$

Measures:

- **Kurtosis:** $\text{kurt}(y) = E[y^4] - 3(E[y^2])^2$
- **Negentropy:** $J(y) = H(y_{\text{gauss}}) - H(y)$

5 Association Rule Mining: Find Patterns [Medium]

5.1 Apriori: Frequent Itemsets [Medium]

Intuition

"People who bought X also bought Y." Find combinations that appear together frequently in transactions.

Everyday Analogy

Grocery Store Layout:

- Discover: Milk and bread often bought together
- Action: Place them far apart (force customers to walk past other items)
- Or: Bundle them for discount

The Math (Step-by-Step)

Key Metrics:

Support (frequency):

$$\text{Support}(X) = \frac{\# \text{ transactions containing } X}{\text{Total transactions}}$$

Confidence (reliability):

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Lift (interest):

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)} = \frac{P(X \cap Y)}{P(X)P(Y)}$$

Apriori Principle: If $\{A, B\}$ is infrequent, then $\{A, B, C\}$ is also infrequent.

Worked Example

Market Basket Analysis:

Transactions:

{milk, bread, butter}
{milk, bread}
{milk, eggs}
{bread, butter}
{milk, bread, butter, eggs}

Minimum support = 40% (2 transactions)

Step 1: Find frequent 1-itemsets

- milk: $4/5 = 80\%$ ✓
- bread: $4/5 = 80\%$ ✓
- butter: $3/5 = 60\%$ ✓
- eggs: $2/5 = 40\%$ ✓

Step 2: Find frequent 2-itemsets

- {milk, bread}: $3/5 = 60\%$ ✓
- {milk, butter}: $2/5 = 40\%$ ✓
- {bread, butter}: $3/5 = 60\%$ ✓

Step 3: Find frequent 3-itemsets

- {milk, bread, butter}: $2/5 = 40\%$ ✓

Generate Rules:

- milk $\rightarrow$ bread: Confidence = $3/4 = 75\%$, Lift = $0.75/0.8 = 0.94$
- bread $\rightarrow$ butter: Confidence = $3/4 = 75\%$, Lift = $0.75/0.6 = 1.25$

6 Anomaly Detection: Find Outliers [Medium]

6.1 Z-Score: Statistical Method [Easy]

💡 Intuition

"How many standard deviations away from normal?" Classic statistical approach assuming normal distribution.

📊 The Math (Step-by-Step)

Univariate Z-Score:

$$z = \frac{x - \mu}{\sigma}$$

Flag if $|z| > \text{threshold}$ (typically 2 or 3)

Multivariate (Mahalanobis Distance):

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Why Mahalanobis?

- Accounts for correlations between features
- Dimensionless (scale-invariant)
- If data is Gaussian: $D_M^2 \sim \chi_d^2$

🔗 Worked Example

Server Monitoring:

Metrics: [Response time (ms), CPU usage (%)]

Normal data: $\boldsymbol{\mu} = [50, 30]$, $\boldsymbol{\Sigma} = \begin{bmatrix} 25 & 10 \\ 10 & 9 \end{bmatrix}$

New observation: $\mathbf{x} = [100, 80]$

Z-scores separately:

$$z_{\text{time}} = \frac{100 - 50}{5} = 10, \quad z_{\text{cpu}} = \frac{80 - 30}{3} = 16.7$$

Both extreme!

Mahalanobis distance:

$$D_M = \sqrt{[50, 50] \begin{bmatrix} 25 & 10 \\ 10 & 9 \end{bmatrix}^{-1} \begin{bmatrix} 50 \\ 50 \end{bmatrix}} = \sqrt{[50, 50] \begin{bmatrix} 0.056 & -0.062 \\ -0.062 & 0.156 \end{bmatrix} \begin{bmatrix} 50 \\ 50 \end{bmatrix}} = \sqrt{305} \approx 17.5$$

Critical value for χ_2^2 at 99% = 9.21

$17.5 > 9.21 \rightarrow \text{Anomaly!}$

6.2 Isolation Forest: Isolate Anomalies [Medium]

💡 Intuition

"Anomalies are few and different." Instead of profiling normal data, explicitly isolate anomalies. They require fewer random splits to isolate.

The Math (Step-by-Step)

Algorithm:

1. Build multiple random trees (isolation trees)
2. Randomly select feature and split value
3. Anomalies isolated closer to root (shorter path)
4. Normal points isolated deeper in tree (longer path)

Anomaly Score:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

where $h(x)$ = path length, $c(n)$ = average path length for n samples.

Interpretation:

- $s \approx 1$: Definitely anomaly
- $s < 0.5$: Likely normal
- $s \approx 0.5$: Uncertain

7 Semi-Supervised Learning: Use Unlabeled Data [Advanced]

7.1 Self-Training: Bootstrap [Medium]

Intuition

Train on labeled data, predict on unlabeled, add confident predictions to training set, repeat.

Algorithm Steps

Self-Training Algorithm:

1. Train model M on labeled data L
2. Predict probabilities on unlabeled data U
3. Select subset U_{conf} where $\max P(y|\mathbf{x}) > \tau$ (threshold)
4. Add $(\mathbf{x}, \hat{y})$ to L for $\mathbf{x} \in U_{conf}$
5. Remove U_{conf} from U
6. Repeat until convergence or U is empty

7.2 Co-Training: Two Views [Advanced]

Intuition

Two classifiers trained on different feature sets teach each other. Requires features to be conditionally independent.

The Math (Step-by-Step)

Assumptions:

- Features split into two views: $\mathbf{x} = [\mathbf{x}^1, \mathbf{x}^2]$
- Each view sufficient for classification
- Views conditionally independent given class

Algorithm:

1. Train C_1 on $(\mathbf{x}^1, y)$, C_2 on $(\mathbf{x}^2, y)$
2. C_1 labels unlabeled data for C_2 (most confident)
3. C_2 labels unlabeled data for C_1 (most confident)
4. Retrain both classifiers
5. Repeat

8 Reinforcement Learning: Learn by Doing [Advanced]

8.1 Q-Learning: Value Iteration [Advanced]

Intuition

Learn a "cheat sheet" (Q-table) showing expected future reward for each action in each state.

The Math (Step-by-Step)

Bellman Equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Terms:

- α : Learning rate (how fast to update)
- γ : Discount factor (importance of future rewards)
- r : Immediate reward
- $\max_{a'} Q(s', a')$: Best future value from next state

Exploration vs Exploitation:

- **Exploit**: Choose best known action (probability $1 - \epsilon$)
- **Explore**: Choose random action (probability ϵ)
- ϵ typically decays over time

Worked Example

Grid World (Robot Navigation):

3×3 grid, goal at center (reward +10), walls (reward -1)

Initial Q-table (all zeros):

State	Up	Down	Left	Right
(0,0)	0	0	0	0
(0,1)	0	0	0	0
...	...	...	...	...

Episode: Start at (0,0), $\epsilon = 0.1$ (10% random)

Step 1: Random action (exploration) → Right

- Hit wall, $r = -1$, stay at (0,0)
- Update: $Q((0,0), \text{right}) = 0 + 0.1[-1 + 0.9(0) - 0] = -0.1$

Step 2: Best action (exploitation) → Down

- Move to (1,0), $r = 0$
- Update: $Q((0,0), \text{down}) = 0 + 0.1[0 + 0.9(0) - 0] = 0$

After many episodes, Q-values converge to true expected rewards.

8.2 Policy Gradient: Direct Optimization [Advanced]

Intuition

Instead of learning values, directly learn the policy (what action to take). Adjust parameters to increase probability of high-reward actions.

The Math (Step-by-Step)

REINFORCE Algorithm:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Intuition:

- $\log \pi_{\theta}(a_t | s_t)$: Log probability of taken action
- G_t : Total return from that point
- If G_t is high → increase probability of that action
- If G_t is low → decrease probability

Actor-Critic:

- **Actor:** Policy $\pi_{\theta}(a|s)$ (what to do)
- **Critic:** Value function $V(s)$ (how good is state)
- **Advantage:** $A(s, a) = Q(s, a) - V(s)$ (how good is action vs average)

Quick Reference: Algorithm Selection

Problem	Try First	If That Fails	Key Consideration
Binary Classification	Logistic Regression	SVM, Random Forest	Interpretability vs Accuracy
Multi-class	Random Forest	Gradient Boosting	Number of classes
Text Classification	Naive Bayes	SVM with TF-IDF	Speed vs Accuracy
Regression	Linear/Ridge	Random Forest, XGBoost	Linearity of relationship
High Dimensions	Lasso, Ridge	Elastic Net, PCA+Model	Feature selection needed?
Images/Complex	CNN (Deep Learning)	Transfer Learning	Data size
Time Series	ARIMA	LSTM, Prophet	Seasonality present?
Anomaly Detection	Isolation Forest	One-Class SVM	Label availability

Algorithm	Training Speed	Prediction Speed	Interpretability	Handles Non-linear
Logistic Regression	Fast	Very Fast	High	No
Naive Bayes	Very Fast	Very Fast	High	No
KNN	None	Slow	Medium	Yes (with kernel)
SVM	Medium	Medium	Low	Yes (with kernel)
Decision Tree	Fast	Fast	Very High	Yes
Random Forest	Medium	Fast	Medium	Yes
Linear Regression	Fast	Very Fast	High	No
Ridge/Lasso	Fast	Very Fast	High	No
PCA	Medium	Fast	Medium	N/A

Key Takeaways

💡 Intuition

1. **Start Simple:** Always try linear models first (baseline)
2. **Feature Engineering** often beats algorithm tuning
3. **Understand Bias-Variance:** More complex $\neq$ better
4. **Validate Properly:** Train/Validation/Test splits, cross-validation
5. **Interpretability Matters:** Can you explain your model?

The ML Mindset:

Data → Features → Model → Predictions → Evaluate → Iterate
